

Phase 1

A complete technical blueprint for building the data pipeline, running the gap analysis, and validating the core edge hypothesis before writing a single line of agent code.

GOAL	Prove Vegas vs. Kalshi price gap is real & directional
TIMELINE	Before YC application deadline — July 27, 2026
OUTPUT	Chart + table showing gap size and predictive value
GATE	Gap confirmed → move to Phase 2. Not confirmed → pivot

Table of Contents

1. What Phase 1 Is and Is Not	<i>pg 3</i>
2. Success Criteria	<i>pg 3</i>
3. Data Sources and APIs	<i>pg 4</i>
4. Python Package Stack	<i>pg 5</i>
5. Environment Setup	<i>pg 5</i>
6. Phase 1 Task Checklist	<i>pg 6</i>
7. The Gap Analysis — Step by Step	<i>pg 7</i>
8. Interpreting Results	<i>pg 8</i>
9. Full Application Codebase Framework	<i>pg 9</i>
10. Directory and File Scaffold	<i>pg 11</i>
11. Phase Gate — Go / No-Go Decision	<i>pg 12</i>

1. What Phase 1 Is and Is Not

Phase 1 is a pure data analysis and validation exercise. You are not building agents, not wiring up any trading logic, and not deploying anything live. The single question Phase 1 answers is: **does a meaningful, directionally predictive gap exist between Vegas consensus probabilities and Kalshi implied probabilities on NBA, NFL, and MLB game contracts?**

IS: Data pipeline pulling two price feeds

IS: Statistical gap analysis across historical games

IS: Chart and table output proving or disproving edge

IS: A go/no-go gate before any agent work begins

IS NOT: Agent framework or LLM integration

IS NOT: Live trading or paper trading system

IS NOT: A probability model you build from scratch

IS NOT: Anything requiring Kalshi account or real capital

2. Success Criteria

Phase 1 is complete and successful when all four of the following are true:

- 1 Gap Exists** — Kalshi implied probabilities diverge from Vegas consensus by more than 3% on at least 20% of sampled game contracts.
- 2 Gap is Directional** — When the gap exceeds a defined threshold, Vegas proves to be the more accurate market (i.e. backing the Vegas side wins more often than 50%) across your full sample set.
- 3 Gap is Recurring** — The divergence pattern appears consistently across multiple sports (MLB, NBA, NFL) and across multiple months of historical data — not clustered in a single period.
- 4 Output is Presentable** — You have a clean chart showing gap distribution and a table showing win-rate when trading the Vegas side — both shareable as a single image in a YC application or investor conversation.

3. Data Sources and APIs

The Odds API — Vegas Consensus Lines

Your source for Vegas sportsbook moneyline prices across NBA, NFL, and MLB. Aggregates 50+ bookmakers into a single normalized JSON response. The consensus and Pinnacle-anchored fair odds endpoints give you vig-removed implied probabilities without building vig-removal math yourself.

Field	Value
Base URL	https://api.the-odds-api.com/v4/
Auth	?apiKey=YOUR_KEY query param
Plan for Phase 1	Developer — \$29/month
Historical endpoint	/historical/sports/{sport}/odds
Sports keys	basketball_nba americanfootball_nfl baseball_mlb
Markets needed	h2h (moneyline)
Odds format	american or decimal — convert to implied probability
Date param	ISO 8601 string — returns nearest snapshot
Rate limit	500 requests/month on developer; each call costs varies by markets x regions

Sample API call — NBA historical moneyline:

```
GET https://api.the-odds-api.com/v4/historical/sports/basketball_nba/odds
?regions=us
&markets;=h2h
&oddsFormat;=american
&apiKey;=YOUR_KEY
&date;=2025-11-15T12:00:00Z

# Returns: list of games with moneyline prices from each sportsbook
# at the nearest available snapshot to the given timestamp
```

Kalshi Historical Data — Two Options

Option A — Lychee (recommended for Phase 1). No-code query interface over the full Kalshi dataset since 2021. Export to CSV in seconds. Use this to get your historical Kalshi contract prices fast without dealing with Kalshi API pagination.

URL: <https://lycheedata.com> — free tier available

Export columns needed: ticker, title, yes_price, no_price, volume, close_time, result

Option B — Kalshi REST API (for live Phase 3 pipeline). Use in later phases when you need real-time price feeds. Historical endpoint requires cursor-based pagination — plan for bulk pulls.

Base URL: <https://trading-api.kalshi.com/trade-api/v2/>

Auth: Bearer token via login endpoint

Historical endpoint: GET /markets/{ticker}/history

ZenHodl — Optional Advanced Dataset

Tick-level orderbook data across Kalshi and Polymarket with paired game-state snapshots. Useful if Phase 1 reveals the gap exists but you need deeper microstructure analysis to understand when and why it appears. Not required for Phase 1 completion.

URL: <https://zenhodl.net/products>

4. Python Package Stack

Package	Version	Purpose
requests	latest	HTTP calls to The Odds API and Kalshi API
pandas	>=2.0	Data loading, merging, filtering CSV / JSON
numpy	latest	Probability math, vig removal calculations
matplotlib	latest	Gap distribution charts and calibration plots
seaborn	latest	Styled statistical visualizations
scipy	latest	Statistical significance testing (chi-squared, t-test)
python-dotenv	latest	Load API keys from .env file safely
sqlalchemy	latest	SQLite database ORM for trade log (Phase 2+)
loguru	latest	Clean structured logging across all modules
rich	latest	Pretty terminal output during analysis runs
jupyter	latest	Notebook environment for Phase 1 analysis
pytest	latest	Unit tests for data pipeline and math functions

Install all at once:

```
pip install requests pandas numpy matplotlib seaborn scipy \
python-dotenv sqlalchemy loguru rich jupyter pytest
```

5. Environment Setup

.env file — never commit this to git:

```
ODDS_API_KEY=your_the_odds_api_key_here
KALSHI_EMAIL=your_kalshi_email_here
KALSHI_PASSWORD=your_kalshi_password_here
KALSHI_API_BASE=https://trading-api.kalshi.com/trade-api/v2
ODDS_API_BASE=https://api.the-odds-api.com/v4
DB_PATH=data/trade_log.db
LOG_LEVEL=INFO
```

Python version and virtual environment:

```
python3 -m venv venv
source venv/bin/activate # Mac/Linux
venv\Scripts\activate # Windows
pip install -r requirements.txt
```

Pre-flight Checklist

- The Odds API key obtained and tested with a single curl call
- Lychee account created and NBA/NFL/MLB CSV exported successfully
- Git repo initialized with .gitignore covering .env and /data/*.csv
- Virtual environment activated and all packages installed
- Jupyter notebook launches without errors from project root

6. Phase 1 Task Checklist

SETUP

- Create project directory structure (see Section 10)
- Initialize git repository and add .gitignore
- Create virtual environment and install all packages
- Create .env file with API keys
- Confirm .env is listed in .gitignore before first commit

DATA — KALSHI

- Log in to Lychee and connect Kalshi Historical integration
- Query NBA game contracts from Oct 2024 – Apr 2025 (full season)
- Query NFL game contracts from Sep 2024 – Feb 2025 (full season)
- Query MLB game contracts from Apr 2025 – Sep 2025 (full season)
- Export each to CSV: kalshi_nba.csv, kalshi_nfl.csv, kalshi_mlb.csv
- Inspect CSVs — confirm yes_price, close_time, and result columns exist
- Convert yes_price (0–100 cents) to implied probability (divide by 100)

DATA — VEGAS

- Sign up for The Odds API developer plan (\$29/month)
- Test API key with a single live request
- Write scripts/fetch_odds_history.py to pull historical snapshots
- Pull NBA moneylines for Oct 2024 – Apr 2025 at game-open timestamps
- Pull NFL moneylines for Sep 2024 – Feb 2025 at game-open timestamps
- Pull MLB moneylines for Apr 2025 – Sep 2025 at game-open timestamps
- Convert American odds to implied probability using vig-removal formula
- Save cleaned outputs: vegas_nba.csv, vegas_nfl.csv, vegas_mlb.csv

MERGE & CLEAN

- Write scripts/merge_feeds.py to join Kalshi and Vegas data on game date + teams
- Handle team name normalization (Vegas uses full names, Kalshi may use abbreviations)
- Drop any games where either feed is missing a price
- Add gap column: $\text{gap} = \text{kalshi_prob} - \text{vegas_prob}$
- Add direction column: positive gap = Kalshi prices home team higher than Vegas
- Save merged output: data/merged_gap_data.csv

ANALYSIS

- Open notebooks/phase1_gap_analysis.ipynb
- Plot histogram of gap distribution across all games
- Calculate mean gap, median gap, and standard deviation
- Set threshold: flag all games where $\text{abs}(\text{gap}) > 0.03$ (3%)

- For flagged games, calculate win rate when trading the Vegas side
- Break results down by sport (NBA vs NFL vs MLB)
- Calculate statistical significance using chi-squared or binomial test
- Identify top gap profiles: which game types show the largest gaps

OUTPUT

- Generate gap_distribution_chart.png — histogram of all gaps
- Generate win_rate_by_threshold.png — win rate at various gap thresholds
- Generate sport_breakdown_table.png — results per sport
- Write a 1-page summary: analysis/phase1_summary.md
- Document findings: gap size, directional accuracy, statistical confidence

7. The Gap Analysis — Step by Step

Step 1 — Convert odds formats to clean probabilities

Both data sources express prices differently. You need a single probability scale (0.0 to 1.0) for both before any comparison is possible.

```
# Vig removal from American odds (two-sided market)
def american_to_prob(odds: int) -> float:
    if odds > 0:
        return 100 / (odds + 100)
    else:
        return abs(odds) / (abs(odds) + 100)

def remove_vig(home_odds: int, away_odds: int) -> tuple[float, float]:
    raw_home = american_to_prob(home_odds)
    raw_away = american_to_prob(away_odds)
    total = raw_home + raw_away # > 1.0 due to vig
    return raw_home / total, raw_away / total # normalized

# Kalshi: yes_price is in cents (0-100), convert directly
def kalshi_to_prob(yes_price_cents: float) -> float:
    return yes_price_cents / 100.0
```

Step 2 — Merge on game identity

Match Vegas and Kalshi rows using game date and team names. Team name normalization is the biggest practical headache — build a lookup dictionary mapping all known abbreviations and alternate names to a single canonical form.

```
TEAM_MAP = {
    'LA Lakers': 'Los Angeles Lakers',
    'LAL': 'Los Angeles Lakers',
    'GS Warriors': 'Golden State Warriors',
    'GSW': 'Golden State Warriors',
    # ... add all NBA, NFL, MLB teams
}

df['home_team'] = df['home_team'].map(lambda x: TEAM_MAP.get(x, x))
merged = pd.merge(kalshi_df, vegas_df,
    on=['game_date', 'home_team', 'away_team'],
    how='inner')
```

Step 3 — Compute the gap

```
merged['gap'] = merged['kalshi_home_prob'] - merged['vegas_home_prob']
merged['abs_gap'] = merged['gap'].abs()
merged['gap_direction'] = merged['gap'].apply(
    lambda g: 'kalshi_higher' if g > 0 else 'vegas_higher'
)

# Flag games where gap is large enough to be tradeable
THRESHOLD = 0.03 # 3 percentage points
flagged = merged[merged['abs_gap'] >= THRESHOLD].copy()
print(f'Flagged games: {len(flagged)} / {len(merged)} total')
```

Step 4 — Measure directional accuracy

```
# When Kalshi prices home team HIGHER than Vegas, does home team lose more?
# i.e. is Vegas right more often when there's a gap?

flagged['vegas_correct'] = (
    (flagged['gap'] > 0) & (flagged['result'] == 'away_win') |
    (flagged['gap'] < 0) & (flagged['result'] == 'home_win')
)

win_rate = flagged['vegas_correct'].mean()
print(f'Vegas correct rate on flagged games: {win_rate:.1%}')
print(f'Sample size: {len(flagged)} games')

# 50%+ with statistical significance = edge exists
# 55%+ = strong edge worth building around
```

8. Interpreting Results

Vegas correct rate > 55% on flagged games	STRONG SIGNAL	Proceed to Phase 2 immediately. This is your founding edge. Document everything and lead with this chart in your YC application.
Vegas correct rate 51–55% on flagged games	WEAK SIGNAL	Proceed but carefully. Raise the gap threshold — only trade larger gaps. Run more games through analysis to see if statistical significance improves with sample size.
Vegas correct rate ~50% on flagged games	NO SIGNAL	Do not proceed to Phase 2 with this hypothesis. Move immediately to testing Kalshi vs. Polymarket cross-market gaps, or information speed gaps around injury reports.
Gaps exist but are very rare (< 5% of games)	SIZE PROBLEM	The signal may be real but too infrequent to build a fund around. Note the finding and move to supplementary signals that can add contract volume.

Statistical significance note: with 200 flagged games, a win rate of 54% or higher is statistically significant at $p < 0.05$ using a one-sided binomial test. With 100 flagged games, you need 57%+ for the same confidence. Run `scipy.stats.binom_test(wins, n=total, p=0.5, alternative='greater')` and report the p-value in your summary document.

9. Full Application Codebase Framework

This is the scaffold for the entire application — not just Phase 1. Build Phase 1 modules first. Later phases slot into the same structure without reorganization. Every module listed here has a defined role that persists through all phases.

CORE LAYER	Shared utilities used by every module above it
<code>config.py</code>	Loads all env vars via python-dotenv. Single source of truth for API keys, DB path, thresholds. No module imports <code>os.environ</code> directly — everything goes through <code>config</code> .
<code>logger.py</code>	Configures loguru logger with file rotation and structured output. Imported by every module.
<code>db.py</code>	SQLAlchemy engine and session factory. Defines all ORM models: <code>TradeLog</code> , <code>GapRecord</code> , <code>AnalysisResult</code> . Used by Phase 2+ trade logging.
<code>utils.py</code>	Shared math functions: <code>american_to_prob()</code> , <code>remove_vig()</code> , <code>kalshi_to_prob()</code> , <code>compute_edge()</code> , <code>kelly_fraction()</code> . Tested in <code>tests/test_utils.py</code> .
DATA LAYER	Pulls and normalizes raw data from external sources
<code>odds_client.py</code>	Wrapper around The Odds API. Methods: <code>get_historical_odds(sport, date)</code> , <code>get_live_odds(sport)</code> , <code>get_consensus_prob(game_id)</code> . Handles pagination and rate limits.
<code>kalshi_client.py</code>	Wrapper around Kalshi REST API. Methods: <code>get_market_history(ticker)</code> , <code>get_live_markets(sport)</code> , <code>get_contract_price(ticker)</code> . Handles auth token refresh.
<code>data_fetcher.py</code>	Orchestrates <code>odds_client</code> and <code>kalshi_client</code> . Fetches both feeds for a given sport and date range and returns raw DataFrames. Used by both analysis scripts and live agent.
ANALYSIS LAYER	Phase 1 — gap analysis and backtest validation
<code>normalizer.py</code>	Team name normalization. Maintains <code>TEAM_MAP</code> dictionary. Exposes <code>normalize_team(name, sport)</code> returning canonical name. Critical for accurate merging.
<code>merger.py</code>	Joins Kalshi and Vegas DataFrames on <code>game_date</code> + <code>home_team</code> + <code>away_team</code> after normalization. Drops unmatched rows and logs match rate.

gap_calculator.py	Computes gap column, abs_gap, gap_direction, and trade_signal. Applies configurable threshold. Flags tradeable games.
backtest.py	Runs full historical backtest: merge → gap → directional accuracy. Outputs win_rate, sample_size, p_value, and sport breakdowns.

AGENT LAYER	Phase 2+ — LLM research agent and decision logic
research_agent.py	Claude API wrapper. Takes a gap record as input, researches the specific reason the gap exists (injury news, sentiment, lineup), returns structured verdict dict.
decision_engine.py	Applies rules-based decision logic on top of agent verdict. Inputs: gap size, confidence, agent_verdict. Output: action (TRADE / SKIP) with position size.
signal_framework.py	Encodes domain expert knowledge as structured prompts per sport. Loaded by research_agent.py to give Claude sport-specific context during research.

EXECUTION LAYER	Phase 2+ — paper and live trade placement
paper_trader.py	Simulates trade placement. Records every decision to TradeLog DB with full reasoning chain, gap size, implied edge, timestamp, and sport.
live_trader.py	Phase 3+. Wraps Kalshi MCP agentic trading API. Places real orders. Never called unless LIVE_MODE=true in .env. Hard-gated.
position_sizer.py	Calculates position size using Kelly Criterion fraction. Inputs: edge, bankroll, max_position_pct. Phase 1 stub only.

DASHBOARD LAYER	Phase 2+ — monitoring and visualization
dashboard.py	Single-page Streamlit or Dash app. Displays: live paper trade log, running P&L, gap detection rate, model calibration curve, win rate by sport. Demo tool for YC interview.
charts.py	Shared matplotlib/seaborn chart generation functions. Used by both Phase 1 analysis notebooks and Phase 2+ dashboard.
reporter.py	Generates daily summary email or Slack message with trade log stats. Phase 3+ feature.

10. Directory and File Scaffold

Copy this structure exactly into Claude Code when scaffolding the repo. Files marked with [P1] are built in Phase 1. Files marked [P2+] are stubbed out in Phase 1 and built in later phases.

```
prediction-market-fund/

■■■ .env # API keys – never commit
■■■ .gitignore # Include .env, /data/*.csv, /venv
■■■ requirements.txt # All pip packages
■■■ README.md # Project overview
■

■■■ config/
■ ■■■ config.py # [P1] Env var loader
■

■■■ core/
■ ■■■ __init__.py
■ ■■■ logger.py # [P1] Loguru setup
■ ■■■ db.py # [P2+] SQLAlchemy models
■ ■■■ utils.py # [P1] Math helpers
■

■■■ data/
■ ■■■ clients/
■ ■ ■■■ __init__.py
■ ■ ■■■ odds_client.py # [P1] The Odds API wrapper
■ ■ ■■■ kalshi_client.py # [P1] Kalshi API wrapper
■ ■■■ fetcher.py # [P1] Orchestrates both clients
■ ■■■ raw/ # CSVs from Lychee and Odds API
■ ■■■ kalshi_nba.csv
■ ■■■ kalshi_nfl.csv
■ ■■■ kalshi_mlb.csv
■ ■■■ vegas_nba.csv
■ ■■■ vegas_nfl.csv
■ ■■■ vegas_mlb.csv
■

■■■ analysis/
■ ■■■ __init__.py
■ ■■■ normalizer.py # [P1] Team name mapping
■ ■■■ merger.py # [P1] Join Kalshi + Vegas feeds
```

```
■ ■■■ gap_calculator.py # [P1] Gap computation
■ ■■■ backtest.py # [P1] Full backtest runner
■ ■■■ phase1_summary.md # [P1] Written findings doc
■
■■■ agent/
■ ■■■ __init__.py
■ ■■■ research_agent.py # [P2] Claude API wrapper
■ ■■■ signal_framework.py # [P2] Expert knowledge prompts
■ ■■■ decision_engine.py # [P2] Rules-based trade logic
■
■■■ execution/
■ ■■■ __init__.py
■ ■■■ paper_trader.py # [P2] Paper trade logger
■ ■■■ position_sizer.py # [P2] Kelly sizing
■ ■■■ live_trader.py # [P3] Real execution – gated
■
■■■ dashboard/
■ ■■■ __init__.py
■ ■■■ app.py # [P2] Streamlit dashboard
■ ■■■ charts.py # [P1] Chart generation helpers
■ ■■■ reporter.py # [P3] Daily summary
■
■■■ notebooks/
■ ■■■ phase1_gap_analysis.ipynb # [P1] Main analysis notebook
■
■■■ scripts/
■ ■■■ fetch_odds_history.py # [P1] CLI: pull Odds API history
■ ■■■ run_backtest.py # [P1] CLI: run full gap analysis
■
■■■ tests/
■■■ __init__.py
■■■ test_utils.py # [P1] Test math helpers
■■■ test_normalizer.py # [P1] Test team name mapping
■■■ test_merger.py # [P1] Test merge logic
```


11. Phase Gate — Go / No-Go Decision

Phase 1 ends with a binary decision. Do not move to Phase 2 until you can answer YES to all of the following questions based on actual data — not intuition, not expectation.

Does the gap exist consistently across all three sports?	NEED: YES	<i>If gaps only appear in one sport, your opportunity is narrower than expected. Still proceed but note the limitation.</i>
Does a gap above 3% occur in at least 15% of all sampled games?	NEED: YES	<i>If gaps are rarer than 10%, contract volume may be too low for a fund. Consider lowering threshold or adding sports.</i>
Is the Vegas-side win rate above 52% on flagged games?	NEED: YES	<i>52% is the minimum profitable threshold at average Kalshi fee structures. Below this is not a fund.</i>
Is the result statistically significant ($p < 0.10$ minimum)?	NEED: YES	<i>Small sample can produce misleading win rates. Statistical significance test is non-negotiable.</i>
Do you have a chart that is explainable in 30 seconds?	NEED: YES	<i>If you cannot explain what the chart shows to a non-technical person in half a minute, it is not ready for YC.</i>

If all five gates pass — move to Phase 2.

Phase 2 is: extract domain expert signal framework, build the research agent, and start the live paper trade log. The gap analysis becomes Slide 1 of your YC application and the founding proof of your entire business.

If any gate fails — run the pivot protocol.

- Test Kalshi vs. Polymarket cross-market gaps on identical contracts
- Test information speed gaps: measure how long injury reports take to price into Kalshi vs. Vegas
- Test sentiment distortion: isolate high-fan-interest games (large market, nationally televised) and measure gap frequency
- Document what failed and why in analysis/phase1_summary.md before pivoting

The edge is somewhere in this market. Phase 1 finds it or tells you exactly where to look next. Either outcome is a good outcome.